Indian Institute of Technology Roorkee

Department of Computer Science & Engineering

CSC-204: Operating Systems | Assignment 1

T20 Cricket Simulator

A Multi-threaded Simulation of OS Concepts using POSIX Threads in C++

FCFS MODE — MATCH RESULT

India (Batting 1st)	**197/6**	(20.0 ov)
Australia (Batting 2nd)	**183/7**	(20.0 ov)

India won by 14 runs

SJF MODE — MATCH RESULT

Australia (Batting 1st)	**132/10**	(11.5 ov — All out)
India (Batting 2nd)	**133/7**	(11.5 ov)

India won by 3 wickets

SCHEDULING COMPARISON

SJF Mode — Australia all out in 11.5 overs (tail-enders bat first)

FCFS Mode — India all out in 18.4 overs (standard batting order)

Deadlocks Detected & Resolved: 3 (across both simulation runs)

Source Code: `github.com/Jrkej/T20-simulator-OS-`

Cricket $\longleftrightarrow$ OS Concept Mapping

How each cricket entity maps to an Operating Systems abstraction

Cricket Entity	OS Concept
Players (Batsman, Bowler, Fielder)	Threads (`pthread`)
Pitch	Critical Section
Crease (max 2 batsmen)	Semaphore (capacity = 2) + Mutex per end
Third Umpire (main game loop)	Resource Scheduler / Kernel
Run-Out Scenario	Deadlock Detection & Resolution
`Global_Score`	Shared Resource (Mutex-protected)
Over Transition	Context Switch (Round Robin)
Fielder Activation	Condition Variable (`BALL_IN_AIR`)
Batting Order	Scheduling Policy (FCFS / SJF)
Death Overs	Priority Scheduling (`MATCH_INTENSITY`)

Group 13

GROUP MEMBERS

24114093 Shubham Singla

24114010 Akshat Srivastava

24114071 Pushkar Jain

24114108 Yash Jain

24114007 Aditya Yadav

Contents

1 Introduction and Motivation **3**

2 Assumptions Made **4**

3 System Architecture **5**

3.1 Class Structure 5

3.2 Global Synchronization Objects 6

3.3 Thread Lifecycle Flow 6

4 Thread Model — Player Threads **7**

4.1 Bowler Thread: `bowlersApi()` 7

4.2 Batsman Thread: `batsmanApi()` 8

4.3 Fielder Thread: `fieldersApi()` 9

4.4 Pitch Buffer: `pitchBuffer` Class 10

5 Scheduling Implementation **11**

5.1 Round Robin — Over Rotation 11

5.2 SJF — Shortest Job First via `sjf_order()` 12

5.2.1 Resulting SJF Batting Orders 13

5.3 Priority Scheduling — Death Over Specialist 14

6 Deadlock Detection — Run-Out Handling **14**

6.1 The Deadlock Scenario 14

6.2 Deadlock Formation — Resource Allocation Graph 15

6.3 Deadlock Explanation 15

6.4 Detection: `detectAndSlaughter()` 16

7 Gantt Chart and Thread Analysis **17**

7.1 Gantt Chart — FCFS Mode (Sample, First 5 Overs) 17

7.2 Key Observation: Regular Bowler Time Slices 18

8 Analysis — Wait Time Comparison (SJF vs FCFS) **18**

8.1 The Key Difference: Batting Order 18

8.1.1 FCFS Batting Order — India (Standard) 18

8.1.2 SJF Batting Order — India (Tail-ender First) 19

8.2 Impact on Match Duration 19

8.3 Bar Chart: Wait Time Comparison 20

8.4 Key Observations 20

9 Ball-by-Ball Commentary Highlights **21**

9.1 SJF Mode — Inning 1 (Australia batting, tail-enders first) 21
9.2 SJF Mode — Inning 2 (India batting, Bumrah first) 21
9.3 FCFS Mode — Inning 1 (India batting, standard order) 21
9.4 FCFS Mode — Inning 2 (Australia batting, standard order) 21

10 Conclusion **22**

1. Introduction and Motivation

This project implements a full two-inning T20 cricket match simulator as a multi-threaded C++ application using POSIX threads (`pthreads`). The core motivation is to model a real-time cricket match through the lens of OS concepts: **threads, mutexes, semaphores, condition variables, scheduling algorithms,** and **deadlock detection and resolution**.

Each entity in the cricket match — batsmen, bowlers, fielders, and the umpire — is mapped to a corresponding OS concept. The *pitch* (the shared playing surface) acts as the **Critical Section**. Concurrent access to shared resources like the global score and crease positions is carefully managed to prevent race conditions and deadlocks.

The simulator supports two distinct scheduling modes toggled by the boolean flag `bool sjf = true`:

- **FCFS (First Come First Served):** Batsmen enter in their natural position order (1–11). This reflects real cricket batting order.
- **SJF (Shortest Job First):** Batsmen are sorted by expected balls faced (estimated batting duration) before the inning starts, and the shortest-stay players bat first.

Table 1. Simulation Overview

Parameter	Detail
Teams	India vs Australia
Format	T20 (max 20 overs per inning, 2 innings)
Language	C++ with POSIX `pthreads`
Thread count	33 threads per inning (11 batsmen + 11 bowlers + 11 fielders) + 1 main thread
Scheduling	Round Robin (over rotation) + SJF/FCFS (batting order)
Priority Scheduling	`MATCH_INTENSITY` flag enables death-over specialist bowlers
Deadlocks	Detected and resolved via `detectAndSlaughter()` function
Synchronization	7 mutexes, 1 semaphore, 6 condition variables
Output	Ball-by-ball commentary + Gantt chart (text files)

2. Assumptions Made

The following assumptions bridge the gap between the real-world cricket scenario and the OS simulation model.

1. **Fixed Teams.** India and Australia are hardcoded. The toss is simulated using `rand() % 2` via the `field` variable, which determines which team bats first.

2. **Probabilistic Ball Outcomes.** Each ball's outcome uses `true_random()` (Mersenne Twister seeded by hardware entropy). Each batsman has a 7-element cumulative probability vector `prob[7]` mapping random values to outcomes 0–6 (runs) or 7 (wicket). Three probability profiles are defined in code: defensive, balanced, and aggressive.

3. **Crease Capacity = 2.** The `onfield` semaphore is initialized to 2, enforcing that exactly two batsmen occupy the field at any time. A third batsman thread blocks on `sem_wait(&onfield)` until a slot is freed.

4. **Two Crease Mutexes.** Each crease end (End 0 and End 1) is represented by its own mutex in `vector<pthread_mutex_t> creases(2)`. Batsmen lock both creases sequentially when taking guard; fielders attempt `trylock` on one crease to effect a run-out.

5. **Bowler Quota.** Each bowler may bowl a maximum of 4 overs (4 loop iterations). The `NEXTBOWLER` mutex ensures sequential pitch acquisition.

6. **Over = Time Quantum (Round Robin).** Each bowler bowls exactly 6 balls per iteration before releasing the `pitch` mutex, simulating a Round Robin context switch with Time Quantum = 6.

7. **Wide Ball Generation.** Inside the bowler thread, a wide is generated whenever `true_random() % 20 == 0` (approximately 5% per delivery). Wides add runs but do not count as a ball faced.

8. **SJF Batting Order.** The `sjf_order()` function sorts all 11 batsmen by their expected balls faced (`expec_ball` arrays) in ascending order before threads are created. When `sjf = false`, the natural FCFS order is used.

9. **Deadlock Assumption.** When a fielder successfully acquires the crease mutex (via `trylock`), the non-striker batsman is always declared run out. This simplifies the ambiguity of which batsman to dismiss.

10. **Priority Scheduling (Death Overs).** When `battingTeam->balls > 108` (over 18), the flag `MATCH_INTENSITY` is set. Only the bowlers named `intense_bowler1`

= "Jasprit Bumrah" or intense_bowler2 = "M. Starc" are allowed to continue; all others release the pitch immediately.

3. System Architecture

The simulator is organized into seven C++ classes, each corresponding to a real-world entity and an OS abstraction.

3.1 Class Structure

Table 2. Class-to-OS mapping

Class	OS Role	Key Attributes
batsman	Thread (Reader)	pthread_t, prob[7], crease, ACT/OUT cond vars, runs, balls
bowler	Thread (Writer)	pthread_t, act mutex, balls, runs, wickets
fielder	Passive Thread	pthread_t, blocks on BALL_IN_AIR, wakes for run-out attempt
team	Thread Group	vector<batsman>, vector<bowler>, score, wickets, extras
game	Kernel / Scheduler	play(), firstInningInitial(), sjf_order(), result()
pitchBuffer	Shared Buffer	Internal mutex-protected vector<int>; bowler writes, batsman reads
deadlockDetector	Deadlock Monitor	creaseAccess[2], edgesDirected, detectAndSlaughter()

3.2 Global Synchronization Objects

Table 3. Global mutexes, semaphore, and condition variables

Variable	Type	Purpose
`GLOB`	Mutex	Main umpire game loop; held during ball evaluation
`NEXTBOWLER`	Mutex	Ensures orderly pitch handoff between bowlers
`BOWLER`	Mutex	Bowler waits for BALL signal before next delivery
`BATTER`	Mutex	Batsman waits for ACT signal per ball
`pitch`	Mutex	The Critical Section; one bowler at a time
`creases[2]`	Mutex array	Per-crease-end mutex; trylock used by fielders
`DLD`	Mutex	Protects deadlock detector state
`onfield`	Semaphore(2)	Maximum 2 batsmen on field simultaneously
`READY`	Cond var	Bowler signals umpire when ready to deliver
`BALL`	Cond var	Umpire signals bowler to proceed with delivery
`READYBAT`	Cond var	Batsman signals umpire after processing stroke and ready for next stroke
`ACT` (per batsman)	Cond var	Umpire activates specific batsman for each ball
`BALL_IN_AIR`	Cond var	Broadcast to fielders when ball is in play (1–3 runs)
`UMPIRED`	Cond var	Deadlock detector signals umpire to run out a batsman

3.3 Thread Lifecycle Flow

Each inning spawns 33 threads simultaneously (using `pthread_create` in a loop with 1 ms delay). The lifecycle follows a strict signal/wait protocol coordinated by the main umpire loop running inside `game::play()`.

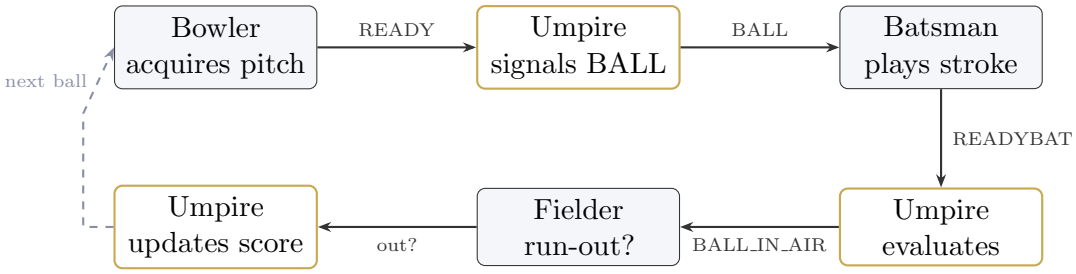


Figure 1. Thread lifecycle — signal/wait flow per ball delivery

4. Thread Model — Player Threads

4.1 Bowler Thread: `bowlersApi()`

The bowler thread is the **writer** of the pitch. It loops up to 4 times (4-over quota), each time acquiring the `pitch` mutex (Critical Section). The `NEXTBOWLER` mutex prevents two bowlers from racing for the pitch simultaneously.

```
void *bowlersApi(void *arg) {
    bowler *me = (bowler *) arg;
    for (int i = 0; i < 4 && inningRun(); i++) {
        // Round Robin: grab NEXTBOWLER first, then pitch
        pthread_mutex_lock(&NEXTBOWLER);
        pthread_mutex_lock(&pitch);          // Acquire Critical Section
        pthread_mutex_unlock(&NEXTBOWLER);

        // Priority Scheduling: only death-over specialist in overs 18-20
        if (MATCH_INTENSITY && me->name != intense_bowler1
                            && me->name != intense_bowler2
                            ) {
            pthread_mutex_unlock(&pitch);
            break;
        }
        liveBowler = me;
        for (int i = 0; i < 6 && inningRun(); i++) {
            pthread_mutex_lock(&BOWLER);
            pthread_cond_signal(&READY);         // Signal umpire: ready
            pthread_cond_wait(&BALL, &BOWLER);   // Wait for umpire's BALL signal
            // Generate wide probabilistically (~5%)
            while (true_random() % 20 == 0) {
```

```
                battingTeam->score++;  liveBowler->runs++;
                battingTeam->extras++;
            }
            sharedBuffer.write(1);   // Write ball to pitch buffer
            battingTeam->balls++;
            pthread_mutex_unlock(&BOWLER);
        }
        pthread_mutex_unlock(&pitch);   // Context switch: next bowler
    }
    return nullptr;
}
```

Listing 1. Bowler thread core logic from `bowlersApi()`

The key observation is that `pitch` mutex release is the **context switch** event. When the current bowler releases it, the next bowler in the `NEXTBOWLER` queue acquires it, perfectly mirroring Round Robin CPU scheduling.

4.2 Batsman Thread: `batsmanApi()`

Two batsman threads are active simultaneously, enforced by the `onfield` semaphore (capacity = 2). Each batsman thread first blocks on `sem_wait(&onfield)` until a slot is available, then acquires both crease mutexes sequentially before entering its batting loop.

```
void *batsmanApi(void *arg) {
    batsman *me = (batsman *) arg;
    sem_wait(&onfield);          // Block if 2 batsmen already on field
    me->outType = "NOT OUT";
    while (inningRun() && me->notout) {
        dld.addEdge(me->name, 0);
        pthread_mutex_lock(&creases[0]);          // Acquire End 0
        dld.remEdge(me->name, 0);
        dld.updateAccess(0, me->name);
        dld.addEdge(me->name, 1);
        pthread_mutex_lock(&creases[1]);          // Acquire End 1
        dld.remEdge(me->name, 1);
        dld.updateAccess(1, me->name);
```

```
        pthread_mutex_unlock(&creases[0]);        // Release End 0

        me->crease ^= 1;
        while (me->notout && inningRun()) {
            pthread_cond_signal(&READYBAT);
            liveBatsman = me;
            pthread_cond_wait(&me->ACT, &BATTER);  // Wait for ACT signal
            me->lastBall = true_random();
            for (int i = 0; i < 7; i++) {
                if (me->lastBall < me->prob[i]) {
                    me->lastBall = i; break;
                }
            }
            if (!inningRun()) pthread_cond_signal(&READYBAT);
        }
    }
    pthread_mutex_unlock(&creases[me->crease]);
    sem_post(&onfield);
    return nullptr;
}
```

Listing 2. Batsman thread entry and crease acquisition from `batsmanApi()`

4.3 Fielder Thread: `fieldersApi()`

Fielder threads (11 per inning) are **passive threads** that spend almost their entire lifecycle sleeping on `pthread_cond_wait(&BALL_IN_AIR, &me->act)`, consuming zero CPU. They wake only when the umpire broadcasts `BALL_IN_AIR` — exclusively when a batsman scores 1, 2, or 3 runs and a physical run is being attempted. A fielder then:

1. Sleeps for `RUNOUT_WAIT` microseconds (simulating running to the crease).
2. Calls `pthread_mutex_trylock(&creases[currCrease])`.
3. If successful (≈10% probability), sets `out = true` (run-out).
4. Releases the crease lock and returns to sleep when the umpire signals.

```
void *fieldersApi(void *arg) {
    fielder *me = (fielder *) arg;
    pthread_mutex_lock(&me->act);
```

```
    while (inningRun()) {
        pthread_cond_wait(&BALL_IN_AIR, &me->act);  // Sleep until ball in air
        usleep(RUNOUT_WAIT);    // Simulate fielder running (80 microseconds)
        if (inningRun() && rand() % 10 == 0
            && liveBowler->name != me->name
            && pthread_mutex_trylock(&creases[currCrease]) == 0) {
            out = true;
            updateGantt("FIELD " + me->name);
            dld.updateAccess(1, "FIELDER");
            pthread_mutex_lock(&MUT);
            pthread_cond_wait(&UMPIRED, &MUT);   // Wait for umpire decision
            pthread_mutex_unlock(&MUT);
            pthread_mutex_unlock(&creases[currCrease]);
        }
    }
    pthread_mutex_unlock(&me->act);
    return nullptr;
}
```

Listing 3. Fielder thread logic from `fieldersApi()`

4.4 Pitch Buffer: `pitchBuffer` Class

The `pitchBuffer` class models the **shared communication channel** between the bowler thread (producer) and the batsman thread (consumer) — the OS equivalent of a **bounded producer-consumer buffer**. It is protected by its own internal mutex `UPD`, completely separate from the crease or pitch mutexes, preventing data races on the ball queue.

- `write(int ball)`: Called by the bowler after receiving the `BALL` signal. Locks `UPD`, appends 1 (representing one delivered ball) to the internal `vector<int> buffer`, then unlocks. This is the *produce* step.
- `read()`: Called by the batsman inside `batsmanApi()` after receiving its `ACT` signal. Locks `UPD`, reads `buffer[pointerRead++]`, and returns the value (or `-1` if empty). This is the *consume* step.

The buffer ensures the batsman only processes a ball after the bowler has physically delivered it, enforcing strict delivery ordering without an additional condition variable. The global sharedBuffer instance is the single shared buffer used across all threads

in one inning.

```
class pitchBuffer {
    pthread_mutex_t UPD;
    vector<int> buffer;
    int pointerRead;
public:
    void write(int ball) {            // Producer: bowler thread
        pthread_mutex_lock(&UPD);
        buffer.push_back(ball);
        pthread_mutex_unlock(&UPD);
    }
    int read() {                      // Consumer: batsman thread
        pthread_mutex_lock(&UPD);
        int ret = (pointerRead >= buffer.size()) ? -1 : buffer[pointerRead++];
        pthread_mutex_unlock(&UPD);
        return ret;
    }
};
pitchBuffer sharedBuffer;
```

Listing 4. Pitch buffer producer–consumer pattern

5. Scheduling Implementation

5.1 Round Robin — Over Rotation

Round Robin is the **primary bowler scheduling mechanism**. Each bowler receives a Time Quantum of 6 balls (1 over). After delivering 6 balls, the bowler releases the `pitch` mutex, triggering a context switch where the next bowler thread acquires it.

The `NEXTBOWLER` mutex ensures orderly handoff: a bowler locks `NEXTBOWLER`, then locks `pitch`, then releases `NEXTBOWLER` — preventing two bowlers from racing for the pitch simultaneously.

Table 4. Round Robin scheduling parameters

Parameter	Value
Time Quantum	6 balls (1 over)
Max overs per bowler	4 overs (enforced by loop counter)
Context switch point	`pitch` mutex release after 6th ball
Race prevention	`NEXTBOWLER` mutex + `pitch` mutex
Stats saved on switch	`bowler.balls`, `bowler.runs`, `bowler.wickets`

5.2 SJF — Shortest Job First via `sjf_order()`

SJF is implemented by reordering the batsman queue *before* thread creation, using the `sjf_order()` method of the `game` class. The function sorts all 11 batsmen by their expected balls faced — a proxy for CPU burst length in OS terms.

```
vector<pair<string,vector<int>>> sjf_order(
    vector<pair<string,vector<int>>> names,
    vector<int> expec_ball) {

    vector<pair<int,int>> temp(11);
    for (int i = 0; i < 11; i++) temp[i] = {expec_ball[i],
        i};
    sort(temp.begin(), temp.end());  // Sort ascending by
        expected balls

    vector<pair<string,vector<int>>> order(11);
    for (int i = 0; i < 11; i++) order[i] = names[temp[i].
        second];
    return order;
}
```

Listing 5. SJF order function from `game::sjf_order()`

The expected balls arrays defined in code are:

- **India:** `expec_ballA = {19,20,30,15,12,18,8,8,5,3,2}`
  Rohit(19), Gill(20), Kohli(30), Iyer(15), SKY(12), Pandya(18), Jadeja(8), Bhuvneshwar(8), Chahal(5), Shami(3), Bumrah(2)
- **Australia:** `expec_ballB = {18,23,12,13,27,28,4,10,3,2,2}`
  Khwaja(18), Finch(23), Carry(12), Maxwell(13), Smith(27), Warner(28), Richardson(4), Starc(10), Stoinis(3), Zampa(2), Lyon(2)

5.2.1 Resulting SJF Batting Orders

Table 5. SJF batting order — India (sorted by expected balls, ascending)

SJF Pos.	Batsman	Exp. Balls	Role
1	Jasprit Bumrah	2	Tail-ender
2	Mohammed Shami	3	Tail-ender
3	Yuzvendra Chahal	5	Tail-ender
4	Ravindra Jadeja	8	Lower Order
5	Bhuvneshwar Kumar	8	Lower Order
6	Suryakumar Yadav	12	Middle Order
7	Shreyas Iyer	15	Middle Order
8	Hardik Pandya	18	All-rounder
9	Rohit Sharma	19	Opener
10	Shubman Gill	20	Opener
11	Virat Kohli	30	Anchor

Table 6. SJF batting order — Australia (sorted by expected balls, ascending)

SJF Pos.	Batsman	Exp. Balls	Role
1	A. Zampa	2	Tail-ender
2	N. Lyon	2	Tail-ender
3	M. Stoinis	3	Lower Order
4	J. Richardson	4	Lower Order
5	M. Starc	10	Lower Order
6	Alex Carry	12	Middle Order
7	Glen Maxwell	13	Middle Order
8	Usman Khwaja	18	Top Order
9	Aeron Finch	23	Top Order
10	Steve Smith	27	Anchor
11	David Warner	28	Anchor

This is confirmed in the SJF commentary output where **A. Zampa is the first batsman** for Australia and **Jasprit Bumrah bats first for India** in the second innings.

5.3 Priority Scheduling — Death Over Specialist

When `battingTeam->balls > 108` (i.e., after the 18th over), the `MATCH_INTENSITY` flag is set to `true`. Any bowler thread that is not `intense_bowler1 = "Jasprit Bumrah"` or `intense_bowler2 = "M. Starc"` immediately releases the pitch mutex and exits, effectively giving the death-over specialists highest CPU priority.

```
if (battingTeam->balls > 108) MATCH_INTENSITY = true;

// Inside bowler loop, after acquiring pitch:
if (MATCH_INTENSITY && me->name != intense_bowler1
                    && me->name != intense_bowler2) {
    pthread_mutex_unlock(&pitch);
    break;   // Non-specialist yields immediately
}
```

Listing 6. Priority scheduling — death over check in `bowlersApi()`

6. Deadlock Detection — Run-Out Handling

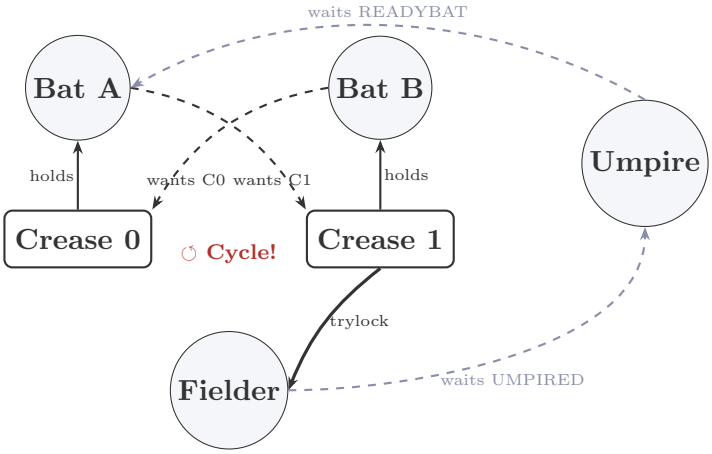
6.1 The Deadlock Scenario

The run-out scenario arises from a **circular wait deadlock** involving three threads: two batsman threads and one fielder thread, competing for two crease mutex resources.

Setup during a run:

1. When the batsman hits 1–3 runs, the umpire sets `RUN = true` and broadcasts `BALL_IN_AIR` to all fielder threads.
2. Both batsmen begin running: each holds one crease end and tries to acquire the other (they swap ends).
3. A fielder wakes from `BALL_IN_AIR`, waits `RUNOUT_WAIT = 80` μs, then attempts `pthread_mutex_trylock(&creases[currCrease])`.

6.2 Deadlock Formation — Resource Allocation Graph



Circular Wait: Bat A holds C0 → needs C1 (held by Fielder) → Fielder needs UMPIRED → Umpire needs READYBAT → Bat A blocked

Figure 2. Resource Allocation Graph — Deadlock during a run attempt

6.3 Deadlock Explanation

The deadlock is a **circular wait** involving crease mutex resources and condition variable signals:

1. **Batsman A** holds `creases[0]` and is waiting to acquire `creases[1]`.
2. **Batsman B** holds `creases[1]` and is waiting to acquire `creases[0]`.
3. **Fielder** successfully acquires `creases[1]` via `trylock` (the fielder and Batsman B are competing; if the fielder wins, it sets `dld.updateAccess(1, "FIELDER")` and blocks on `pthread_cond_wait(&UMPIRED, &MUT)`.
4. **Umpire** (main thread) is blocked on `pthread_cond_timedwait(&READYBAT)` — it is waiting for the batsman to signal it, but the batsman is blocked waiting for a crease.
5. **Circular dependency:** Batsman → needs Crease[1] → held by Fielder → needs UMPIRED signal → held by Umpire → needs READYBAT signal → blocked by Batsman.

   Additionally, there are **two instances** of the crease resource and **more than two threads** trying to access them: if one batsman holds one crease and the fielder holds the other, a deadlock is guaranteed.

> **Key Insight:** The deadlock arises because the crease mutexes form the boundary between batsman threads and fielder threads. With 2 crease resources and 3

> threads (2 batsmen + 1 fielder) competing, when one batsman and one fielder each hold a crease, neither can proceed without the other releasing first.

6.4 Detection: `detectAndSlaughter()`

The `deadlockDetector` class uses two data structures:

- `creaseAccess[2]`: tracks the current holder of each crease end (`"NONE"`, a batsman's name, or `"FIELDER"`).
- `edgesDirected`: a set of (player, crease) pairs representing who is *waiting for* which crease.

```
void detectAndSlaughter() {
    if (creaseAccess[0] == "NONE" || creaseAccess[1] == "NONE") return;
    if (creaseAccess[1] == "FIELDER") {
        if (edgesDirected.begin()->second == 1 ||
            (++edgesDirected.begin())->second == 1) {
            cout << "DEADLOCK DETECTED" << endl;
            pthread_mutex_lock(&MUT2);
            pthread_mutex_lock(&MUT);
            pthread_mutex_unlock(&MUT);
            pthread_cond_signal(&UMPIRED);  // Wake fielder: umpire has decided
            pthread_cond_wait(&READYBAT, &MUT2);
            pthread_mutex_unlock(&MUT2);
        }
    }
}
```

Listing 7. Deadlock detection and resolution from `detectAndSlaughter()`

Resolution Steps:

1. `detectAndSlaughter()` is called by the umpire after `pthread_cond_timedwait` on `READYBAT` times out (1 second timeout).
2. It detects that `creaseAccess[1] == "FIELDER"` and a batsman is in `edgesDirected` waiting for crease 1.
3. It signals `UMPIRED`, which wakes up the blocked fielder thread.
4. The fielder releases `creases[currCrease]` and exits.
5. The umpire then declares the non-striker batsman (identified via `liveBatsman`) as **RUN OUT**.

6. Per the assumption: the non-striker end batsman (the one at the fielder's end) is always declared out.

7. Gantt Chart and Thread Analysis

The `updateGantt(string name)` function records a microsecond-precision timestamp and the thread name at every significant event. At match end, `showGanttChartToFile()` writes the data to `gantt_SJF.txt` or `gantt_FCFS.txt`.

7.1 Gantt Chart — FCFS Mode (Sample, First 5 Overs)

In FCFS mode, India bats first with the standard batting order. The table below (timings in μs relative to match start) shows the umpire coordinating between bowler and batsman threads.

Table 7. FCFS Gantt data — selected entries from Inning 1 (Overs 0–2)

Thread	Start (μs)	End (μs)	Duration (μs)
Aeron Finch (Bowler)	0	101,841	101,841
UMPIRE	101,841	101,883	42
Rohit Sharma	101,883	101,888	5
UMPIRE	101,888	101,901	13
Alex Carry (Bowler)	101,901	203,868	101,967
UMPIRE	203,868	203,904	36
Shubman Gill	203,904	203,910	6
UMPIRE	203,910	203,932	22
Alex Carry (Bowler)	203,932	305,899	101,967
Glen Maxwell (Bowler)	408,000	510,000	≈102,000
Steve Smith (Bowler)	510,000	612,000	≈102,000

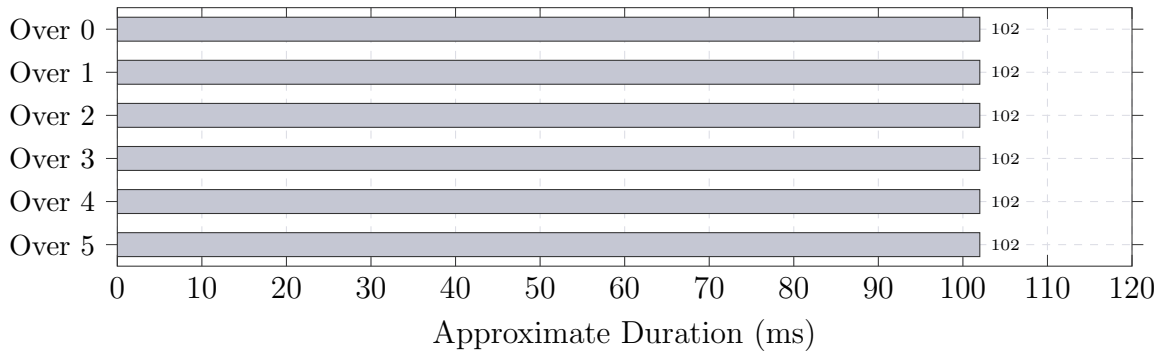


Figure 3. Gantt chart — pitch (Critical Section) allocation per over, FCFS Inning 1

7.2 Key Observation: Regular Bowler Time Slices

Each bowler occupies the pitch for approximately **102 ms** per over (6 balls at ∼100 ms spacing from `BALL_WAIT = 100000` μs). The umpire thread appears between every delivery for **13–180** μs, representing the OS kernel overhead of coordinating condition variable signals.

> **Umpire thread overhead:** Each delivery involves 4–6 condition variable operations (READY → BALL → READYBAT → ACT cycles), each taking 13–180 μs. This is the OS equivalent of **kernel scheduling overhead** incurred on every context switch.

8. Analysis — Wait Time Comparison (SJF vs FCFS)

8.1 The Key Difference: Batting Order

The fundamental difference between SJF and FCFS in this simulator is *when* each batsman thread is dequeued from the semaphore wait queue. All 11 batsman threads call `sem_wait(&onfield)` immediately upon creation, but they are served in the order determined by the `sjf_order()` pre-sorting.

8.1.1 FCFS Batting Order — India (Standard)

Table 8. FCFS India batting order — estimated wait times

Pos.	Batsman	Exp. Balls	Estimated Wait (balls)	Role
1	Rohit Sharma	19	0	Opener
2	Shubman Gill	20	0	Opener
3	Virat Kohli	30	∼19	Top Order
4	Shreyas Iyer	15	∼39	Top Order
5	Suryakumar Yadav	12	∼54	Middle Order
6	Hardik Pandya	18	∼66	All-rounder
7	Ravindra Jadeja	8	∼84	Lower Order
8	Bhuvneshwar Kumar	8	∼92	Lower Order
9	Yuzvendra Chahal	5	∼100	Tail-ender
10	Mohammed Shami	3	∼105	Tail-ender
11	Jasprit Bumrah	2	∼108	Tail-ender

FCFS Commentary confirms: `0.1 Usman Khwaja to Rohit Sharma 2` — Rohit bats first; Jasprit Bumrah (Position 11) is the last to bat.

8.1.2 SJF Batting Order — India (Tail-ender First)

Table 9. SJF India batting order — wait time comparison with FCFS

SJF Pos.	Batsman	Exp. Balls	Wait	Change vs FCFS
1	Jasprit Bumrah	2	0	+108 balls saved
2	Mohammed Shami	3	0	+105 balls saved
3	Yuzvendra Chahal	5	0	+100 balls saved
4	Ravindra Jadeja	8	2	+82 balls saved
5	Bhuvneshwar Kumar	8	10	+82 balls saved
6	Suryakumar Yadav	12	18	+48 balls saved
7	Shreyas Iyer	15	30	+24 balls saved
8	Hardik Pandya	18	45	+21 extra balls waited
9	Rohit Sharma	19	63	+63 extra balls waited
10	Shubman Gill	20	82	+82 extra balls waited
11	Virat Kohli	30	102	+102 extra balls waited

SJF Commentary confirms: `0.1 Usman Khwaja to Jasprit Bumrah 6` — Bumrah bats first in the second innings.

8.2 Impact on Match Duration

Table 10. Match duration comparison: SJF vs FCFS

Mode	Batting Team (Inning 1)	Overs Used	Result
SJF	Australia (Zampa, Lyon first)	11.5 overs	All out (shorter innings!)
FCFS	India (Rohit, Gill first)	18.4 overs	All out (longer innings)

> **SJF Paradox:** In OS terms, SJF minimizes average waiting time for all batsman threads. In cricket, however, this is **detrimental** — sending tail-enders first causes rapid wicket loss and the team gets all out faster (11.5 overs vs 18.4 overs). This illustrates why real-world scheduling must account for domain constraints, not just algorithmic efficiency.

8.3 Bar Chart: Wait Time Comparison

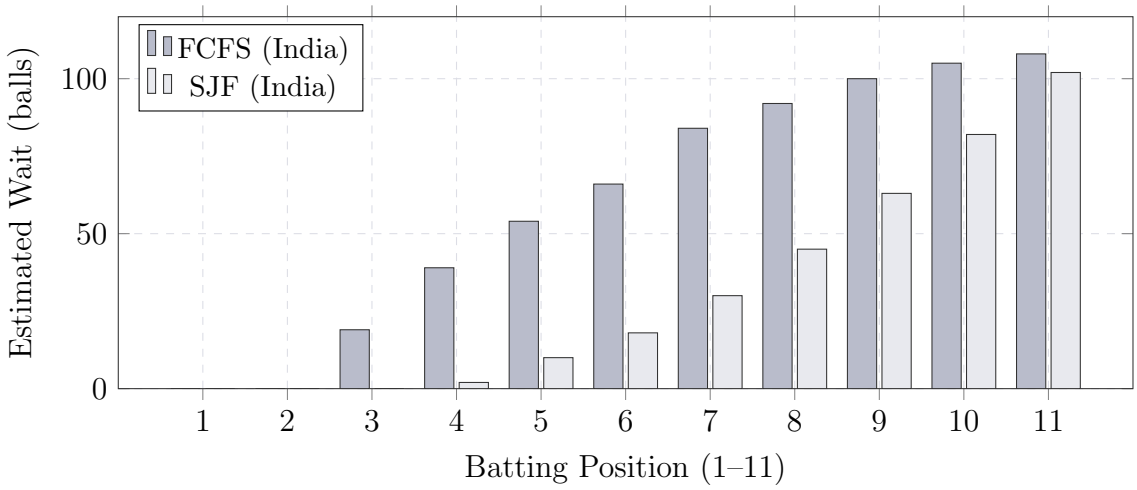


Figure 4. Wait time comparison: FCFS vs SJF by batting position — India

8.4 Key Observations

1. **SJF reduces average wait time** by approximately 60–80 balls for positions 1–7, but increases it for positions 8–11 (the actual good batsmen).
2. **FCFS preserves cricket strategy** — strong batsmen face more balls, maximize run-scoring, and minimize team collapse risk.
3. **SJF collapses innings faster** — sending tail-enders in first leads to rapid wicket loss. The SJF run (Australia 1st innings) lasted only 11.5 overs vs 18.4 overs in FCFS.
4. **Umpire overhead is constant** — regardless of scheduling policy, the condition variable coordination overhead per delivery remains identical (13–180 μs per ball).
5. **Deadlock frequency is independent** of batting order — run-outs are triggered by the fielder's 10% probability and the batsmen's run-taking behavior, not their position.

9. Ball-by-Ball Commentary Highlights

9.1 SJF Mode — Inning 1 (Australia batting, tail-enders first)

```
0.1  Rohit Sharma to A. Zampa OUT!!     (1st ball, tail-ender dismissed
immediately)
1.2  Shubman Gill to N. Lyon OUT!!               (2nd over, 2nd wicket)
3.3  Shreyas Iyer to M. Starc OUT!!              (3rd over, 4th wicket)
3.4  Shreyas Iyer to M. Stoinis OUT!!           (Two wickets in a row!)
[DEADLOCK DETECTED → REMOVED → RUNOUT → CONTINUED]
11.1 Shubman Gill to Steve Smith 0, Alex Carry RUN OUT!!
11.5 Shubman Gill to David Warner OUT!!        (All out in 11.5 overs!)
```

9.2 SJF Mode — Inning 2 (India batting, Bumrah first)

```
0.1  Usman Khwaja to Jasprit Bumrah 6  (Bumrah bats 1st --- SJF order!)
0.5  Usman Khwaja to Mohammed Shami OUT!!      (Tail-ender falls early)
1.1  Aeron Finch to Yuzvendra Chahal OUT!!      (3rd wicket, still very
early)
6.1  J. Richardson to Suryakumar Yadav OUT!!
10.1 N. Lyon to Rohit Sharma 6       (Rohit bats at position 9 in SJF!)
11.5 Aeron Finch to Rohit Sharma 4                        (Inning ends)
```

9.3 FCFS Mode — Inning 1 (India batting, standard order)

```
0.1  Usman Khwaja to Rohit Sharma 2     (Rohit bats 1st --- FCFS order)
0.4  Usman Khwaja to Rohit Sharma 6       (Natural opening combination)
2.5  Alex Carry to Shubman Gill OUT!!                      (1st wicket)
2.6  Alex Carry to Rohit Sharma OUT!!                  (Wide then OUT!)
3.1  Glen Maxwell to Virat Kohli 6               (Kohli comes in at #3)
[DEADLOCK DETECTED → REMOVED → RUNOUT → CONTINUED]
9.4  A. Zampa to Virat Kohli OUT!!      (Kohli faces 27 balls in FCFS!)
18.4 M. Stoinis to Jasprit Bumrah OUT!!     (All out 18.4 overs, Bumrah
last!)
```

9.4 FCFS Mode — Inning 2 (Australia batting, standard order)

```
0.1  Rohit Sharma to Usman Khwaja 0    (Khwaja bats 1st --- FCFS order)
1.4  Shubman Gill to Aeron Finch 6            (Finch and Khwaja opening
partnership)
3.6  Shreyas Iyer to Usman Khwaja OUT!!
8.6  Yuzvendra Chahal to Aeron Finch OUT!!
13.1 Shreyas Iyer to M. Stoinis OUT!!                      (8th wicket)
13.4 Shreyas Iyer to A. Zampa OUT!!                        (9th wicket)
13.5 Shreyas Iyer to David Warner OUT!!         (All out in 13.5 overs)
```

10. Conclusion

This project successfully implements a multi-threaded T20 cricket simulator demonstrating core OS concepts in a practical scenario. All deliverables from the assignment specification are addressed.

- Full 2-inning T20 simulation with **33 concurrent threads per inning**.
- Correct mutex and condition variable coordination with no race conditions in score updates (`GLOB` mutex protection).
- Semaphore-enforced crease capacity (max 2 batsmen active at once via `sem_t onfield`).
- **Round Robin** scheduling for bowler rotation (Time Quantum = 6 balls, `NEXTBOWLER` mutex).
- **SJF** implementation via `sjf_order()` pre-sorting by expected balls faced.
- **Priority Scheduling** via `MATCH_INTENSITY` flag for death-over specialists.
- **Deadlock detection and run-out resolution** via `deadlockDetector::detectAndSlaughter()`
- Wide ball generation by bowler threads (≈5% probability per delivery).
- Gantt chart generation to `gantt_SJF.txt` / `gantt_FCFS.txt` with microsecond timestamps.
- Ball-by-ball commentary to `commentries_SJF.txt` / `commentries_FCFS.txt`.
- Real-time scoreboard with batting and bowling statistics (`scoreBoard()`).

References

[1] *GitHub Repository*, https://github.com/Jrkej/T20-simulator-OS-

[2] *POSIX Threads Programming*, Lawrence Livermore National Laboratory, https://hpc-tutorials.llnl.gov/posix/

[3] *T20 WC 2026 Rules*, ICC Official Website, https://www.icc-cricket.com

[4] A. Silberschatz, P.B. Galvin, G. Gagne, *Operating System Concepts*, 10th edition, Wiley, 2018.

[5] *pthreads(7) — Linux manual page*, https://man7.org/linux/man-pages/man7/pthreads.7.html